

Checklist de Blockchain — Trabajo Final SDyPP

1. Funciones de Blockchain requeridas

- Nodos con clave pública y privada.
- Pool de minado: modo cooperativo y competitivo, y condiciones de validación del ganador.
 - Worker CPU en funcionamiento.
 - Worker GPU en funcionamiento.
- ¿Cómo es el protocolo? Competencia y/o coordinación.
- Detalle de cómo "configuraron" arquitectura CUDA / versiones.
- Manejo de fallas en workers: ¿Qué pasa si uno cae? ¿Se reasignan tareas?
- Keep-alive de mineros GPU hacia el pool de transacciones: el sistema conoce la capacidad de procesamiento disponible.
- Fallback ante ausencia de GPUs: reducción de la complejidad del prefijo y/o creación dinámica de mineros CPU (HPA, VMs on-demand o Cloud Run jobs).
- Métricas por tipo de recurso. Algunas a considerar pueden ser:
 - Tasa de éxito CPU vs GPU.
 - Hashes por segundo (por nodo).
 - Tiempos de minería por prefijo.
 - Latencia entre RabbitMQ y worker.
 - Tiempo de validación del bloque.

2. Configuración de Plataforma escalable

Servicios distribuidos en funcionamiento sobre Kubernetes:

- Base de datos.
- Sistema de colas.
- Secretos.
- Configuraciones (ConfigMaps).
- Certificados (HTTPS).
- Plataforma de Logging (Colector de N servicios y M réplicas).
- Monitoreo (alertas, dashboards).
- Sincronización de relojes con NTP en los componentes.
- Endpoint público de estado (health/status) por servicio, accesible desde Internet. No requiere GUI: puede devolver un JSON (key = servicio, value = status).

3. Configuraciones para ambiente productivo real

Para cada ítem, en el informe, detalle y describa cómo lo llevaron a cabo:

- Autoscaler activado por uso de CPU (Cluster Autoscaler / Karpenter).
- HPA por métricas "comunes" o "específicas".
- Servicios definidos mínimamente como StatefulSet para escalar con los PVC.
- Limitación de recursos y securityContext para contenedores (no root, solo lo necesario).
- Configuración de tolerations/affinity/nodeSelector para separar workloads de infraestructura (Redis/RabbitMQ/Secrets Operator) de los de minería.
- Seguridad y segmentación de servicios:
 - Uso de namespaces.
 - RBAC en Kubernetes.
 - Canal seguro entre nodos (TLS entre servicios: coordinador, workers, Redis, RabbitMQ).
 - Zero static keys: autenticación contra el cloud provider vía Workload Identity / OIDC (sin llaves estáticas en el repo ni en los pipelines).
 - Registros Docker: image pull secrets o Workload Identity (no enviar credenciales en payloads).
 - Registros de actividades (logs) gestionados en memoria y disco.

4. Pruebas del sistema

- Pruebas unitarias y de integración automatizadas que cubran las funcionalidades críticas del proyecto.
- Prueba del sistema con distintos casos de transacciones y carga:
 - N transacciones con M recursos corriendo.
 - N transacciones con 2xM recursos corriendo.
 - Otras configuraciones.
- Rangos de referencia según enunciado:
 - Bulks de transacciones: de 1 a 100.000.
 - Dificultad de prefijo de hash: de 1 a 8 caracteres.
 - Fragmentación del pool de transacciones: de 1% a 50%.
 - Ingreso y egreso de nodos GPU (verificar generación dinámica de nodos CPU cuando sea necesario).

5. Pipelines de despliegue

- Pipeline 1 — Despliegue de infra básica (Terraform/OpenTofu o similar).
 - Mencionar claramente cómo es la configuración de los secretos para habilitar despliegues 2 a N.

- Pipeline 2 — Despliegue de servicios core (BD, colas, etc.).
- Pipeline 3-N — Despliegue de apps.
- **Si aplica**, pipeline de VMs externas al clúster: despliegue de máquinas virtuales que actúan como nodos de trabajo adicionales, ajustadas dinámicamente según la demanda de la red.
- Gitleaks (o similar) integrado en el pipeline de CI: si detecta un secret hardcodeado, el pipeline debe fallar.

6. Repositorio y entrega

- Repositorio público (GitHub, Bitbucket o GitLab) con una carpeta y un README.md por pilar.
- Cada README.md incluye como mínimo: instrucciones para ejecutar el proyecto, diagrama de arquitectura y decisiones de diseño tomadas.
- No hay .env, credenciales ni secrets commiteados; .gitignore apropiado desde el inicio. Si se expuso un secret, fue revocado y regenerado.
- Aplicación compilada para ejecución desde la terminal, con recursos listos para desplegar sin abrir un IDE.
- Grabación de video subida al repositorio explicando servicios, componentes y configuraciones (debe demostrar comprensión de cada punto).
- Herramientas de IA utilizadas en el ciclo de desarrollo (Cursor, ChatGPT/Codex, Claude, Copilot, etc.): declarar cuáles y cómo ayudaron.

7. Informe

- Comparativa y análisis de resultados.
- Diagrama de arquitectura.
- Explicar cómo funciona el pool de transacciones y cómo escala con diversas cargas y workers.
- Mostrar cómo el sistema se comporta con diversos casos de prueba:
 - N transacciones con M recursos corriendo.
 - N transacciones con 2xM recursos corriendo.
 - Otras configuraciones.
- Gráficos comparativos de los tiempos de respuesta medidos para cada configuración de prueba.
-
- Reflexión crítica: limitaciones actuales, posibles mejoras, contexto real donde aplicarías esta solución.